Демидовский университет

Позиционирование и перемещение виджетов

Автор: доцент, кандидат технических наук
Ксения Владимировна Лагутина

Контейнер для виджетов

- Контейнеры Qt – это простой и мощный способ автоматического размещения дочерних виджетов
- Контейнеры автоматически позиционируют и изменяют размер виджетов при изменении объема доступного для них пространства
  - Гарантируют, что виджеты расположены последовательно и пользовательский интерфейс остается пригодным для использования

•setLayout() – добавить контейнер в родительский виджет

https://doc.qt.io/qtforpython-6/overviews/qtwidgets-layout.html

Демидовский университет

Основные контейнеры

- QLayout – базовый класс
  - QBoxLayout выстраивает дочерние виджеты по горизонтали или вертикали
    - QHBoxLayout выстраивает виджеты горизонтально
    - QVBoxLayout выстраивает виджеты вертикально
  - QFormLayout управляет формами виджетов ввода и связанными с ними метками
  - QGridLayout размещает виджеты в виде сетки
  - QStackedLayout – стек виджетов, в котором одновременно виден только один виджет
- QGraphicsAnchorLayout – компоновщик, в котором можно привязывать виджеты друг к другу
- QButtonGroup – контейнер для кнопок
- QGroupBox – рамка группы с заголовком

Демидовский университет

QLayout — базовый класс для контейнеров

- addWidget()/removeWidget() – добавить/удалить виджет
- setSizeConstraint() – способ управления размером
  - QLayout.SetDefaultConstraint – минимальный размер основного виджета minimumSize() или указанный
  - QLayout.SetFixedSize – размер основного виджета sizeHint(), его нельзя изменить
  - QLayout.SetMinimumSize – минимальный размер основного виджета не меньше minimumSize()
  - QLayout.SetMaximumSize – максимальный размер основного виджета не больше maximumSize()
  - QLayout.SetMinAndMaxSize –установлены минимум и максимум размеров
  - QLayout.SetNoConstraint – нет ограничений
- setMenuBar() – меню
- hasHeightForWidth() и heightForWidth() – настройка высоты, зависящей от ширины

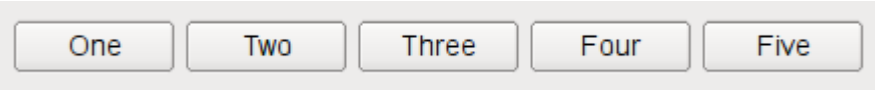
Демидовский университет

Процесс добавления виджетов

- Всем виджетам изначально будет выделено определенное количество места в соответствии с sizePolicy() и sizeHint()
- Если для какого-либо из виджетов установлены коэффициенты растяжения, значение которых больше нуля, то ему выделяется пространство пропорционально их коэффициенту растяжения
- Если для какого-либо из виджетов коэффициенты растяжения установлены равными нулю, он получит больше места только в том случае, если другим виджетам это пространство не нужно
- Всем виджетам, которые занимают меньше места, чем их минимальный размер, выделяется требуемый минимальный размер
- Всем виджетам, которые занимают больше места, чем их максимальный размер, выделяется пространство максимального размера, которое им требуется

QBoxLayout — ряд виджетов

- addLayout() – добавить контейнер в контейнер
- addSpacing() – промежуток между виджетами
- addStretch() – пустое растягиваемое пространство
- insert*() – вставка в начало или середину
- setDirection() – направление виджетов вертикально или горизонтально
- Наследники:
  - QHBoxLayout выстраивает виджеты горизонтально
  - QVBoxLayout выстраивает виджеты вертикально

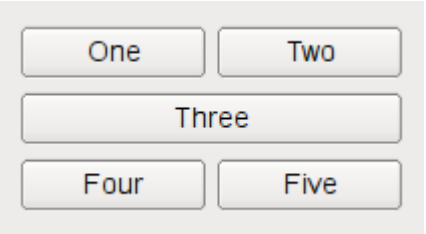


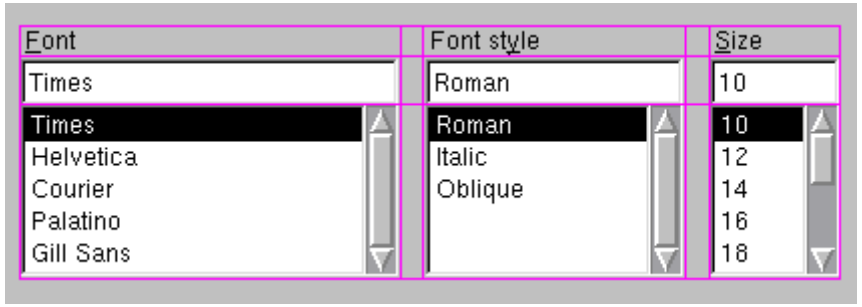
https://doc.qt.io/qtforpython/PySide6/QtWidgets/QBoxLayout.html

Демидовский университет

QGridLayout — сетка

- setColumnMinimumWidth() – минимальная ширина каждого виджета в столбце
- setColumnStretch() – коэффициент растяжения – какую часть доступного пространства займет столбец сверх необходимого минимума
- addItem() и addWidget() – поместить виджет в отдельную ячейку или набор ячеек





https://doc.qt.io/qtforpython/PySide6/QtWidgets/QGridLayout.html

Демидовский университет

Пример QGridLayout

```
layout = QGridLayout()
layout.addWidget(QLabel('First'), 0, 0)
layout.addWidget(QLabel('Second'), 0, 1)
layout.addWidget(QLabel('Text in two cells'), 1, 0, 1, 2)
layout.addWidget(QLabel('bottom left'), 2, 0)
layout.addWidget(QLabel('bottom right'), 2, 1)
```

Демидовский университет

QFormLayout — управление формой

- Две колонки
- Стилизуется под ОС
- addRow() – добавить строку в форму
- removeRow() – удалить строку
- rowCount() – количество строк
- setRowVisible() – видимость строки
- setFormAlignment() – выравнивание содержимого

https://doc.qt.io/qtforpython/PySide6/QtWidgets/QFormLayout.html

Демидовский университет

Пример QFormLayout

```
button1 = QPushButton("One")
lineEdit1 = QLineEdit()
button2 = QPushButton("Two")
lineEdit2 = QLineEdit()
button3 = QPushButton("Three")
lineEdit3 = QLineEdit()
layout = QFormLayout()
layout.addRow(button1, lineEdit1)
layout.addRow(button2, lineEdit2)
layout.addRow(button3, lineEdit3)
```

Демидовский университет

QStackedLayout — стек виджетов

- currentIndex() / currentWidget () – верхний виджет
- insertWidget() – вставка в середину
- setStackingMode() – режим стека: виден один элемент или все
- widget() – виджет на запрашиваемой позиции
- currentChanged(index) – изменился верхний виджет
- widgetRemoved(index) – виджет удален

https://doc.qt.io/qtforpython/PySide6/QtWidgets/QStackedLayout.html

Демидовский университет

QStackedWidget — стек виджетов

- Удобный виджет, созданный поверх класса QStackedLayout
- QStackedWidget не предоставляет пользователю встроенных средств для переключения страниц
- Обычно это делается через QComboBox или QListWidget, в котором хранятся заголовки страниц QStackedWidget

https://doc.qt.io/qtforpython/PySide6/QtWidgets/QStackedWidget.html

Демидовский университет

Пример QStackedLayout

```
firstPageWidget = QLabel('First')
secondPageWidget = QLabel('Second')
thirdPageWidget = QLabel('Third')
stackedLayout = QStackedLayout()
stackedLayout.setStackingMode(QStackedLayout.StackAll)
stackedLayout.addWidget(firstPageWidget)
stackedLayout.addWidget(secondPageWidget)
stackedLayout.addWidget(thirdPageWidget)
```

Демидовский университет

QScrollArea — виджет для прокрутки

- Область прокрутки используется для отображения содержимого дочернего виджета внутри фрейма
- Если размер виджета превышает размер фрейма, QScrollArea может предоставить полосы прокрутки, чтобы можно было просмотреть всю область дочернего виджета
- Дочерний виджет должен быть указан с помощью setWidget()
- Элемент прокрутки: QScrollBar — наследник QAbstractSlider
- Политика отображения setHorizontalScrollBarPolicy(), setVerticalScrollBarPolicy():
  - Qt.ScrollBarAsNeeded, Qt.ScrollBarAlwaysOff, Qt.ScrollBarAlwaysOn

https://doc.qt.io/qtforpython/PySide6/QtWidgets/QScrollArea.html

Пример прокрутки

```
class Window(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Пример прокрутки")
        self.setMinimumWidth(100)
        self.setMinimumHeight(100)
        layout = QVBoxLayout()
        imageLabel = QLabel()
        imageLabel.setPixmap(QPixmap("landscape.jpg"))
        scrollArea = QScrollArea()
        scrollArea.setWidget(imageLabel)
        layout.addWidget(scrollArea)
        self.setLayout(layout)
```

Демидовский университет

QTabWidget – вкладки

- addTab() – добавить виджет как вкладку
- removeTab() – удалить вкладку
- count() – количество вкладок
- setMovable(), setTabEnabled(), setTabVisible() – настройка вкладок
- tabBar() – список вкладок как объект
- setElideMode() – способ сокращения текста в заголовках вкладок

https://doc.qt.io/qtforpython/PySide6/QtWidgets/QTabWidget.html

Демидовский университет

QTabBar – заголовки вкладок

- addTab() – добавить текст/иконку как заголовок вкладки
- moveTab(from, to) – переместить вкладку
- removeTab() – удалить вкладку
- count() – количество вкладок
- setMovable(), setTabEnabled(), setTabVisible() – настройка вкладок
- setElideMode() – способ сокращения текста в заголовках вкладок
- Сигналы:
  - currentChanged(index) – открыта другая вкладка
  - tabBarClicked(index) – клик по вкладке
  - tabMoved (from, to) – вкладка в списке смещена

Демидовский университет

https://doc.qt.io/qtforpython/PySide6/QtWidgets/QTabBar.html

Пример работы с вкладками

```
class TabWidget(QTabWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        imageLabel = QLabel()
        imageLabel.setPixmap(QPixmap("landscape.jpg"))
        scrollArea = QScrollArea()
        scrollArea.setWidget(imageLabel)
        self.addTab(scrollArea, "Прокрутка")
        secondTab = QWidget()
        button1 = QPushButton("One")
        lineEdit1 = QLineEdit()
        button2 = QPushButton("Two")
        lineEdit2 = QLineEdit()
        button3 = QPushButton("Three")
        lineEdit3 = QLineEdit()
        layout = QFormLayout()
        layout.addRow(button1, lineEdit1)
        layout.addRow(button2, lineEdit2)
        layout.addRow(button3, lineEdit3)
        secondTab.setLayout(layout)
        self.addTab(secondTab, "Пример формы")
```

Демидовский университет

QToolBox – колонка с вкладками

- addItem(widget, text) – добавить виджет на вкладку
- removeItem(index) – удалить виджет
- count() – количество вкладок
- currentIndex(), currentWidget() – текущая вкладка
- setItemEnabled(index, enabled) – блокировать/сделать доступной вкладку
- widget(index) – вкладка по индексу

https://doc.qt.io/qtforpython/PySide6/QtWidgets/QToolBox.html

Пример колонки

```
class ToolWidget(QToolBox):
    def __init__(self, parent=None):
        super().__init__(parent)
        imageLabel = QLabel()
        imageLabel.setPixmap(QPixmap("landscape.jpg"))
        scrollArea = QScrollArea()
        scrollArea.setWidget(imageLabel)
        self.addItem(scrollArea, "Прокрутка")
        secondTab = QWidget()
        button1 = QPushButton("One")
        lineEdit1 = QLineEdit()
        button2 = QPushButton("Two")
        lineEdit2 = QLineEdit()
        button3 = QPushButton("Three")
        lineEdit3 = QLineEdit()
        layout = QFormLayout()
        layout.addRow(button1, lineEdit1)
        layout.addRow(button2, lineEdit2)
        layout.addRow(button3, lineEdit3)
        secondTab.setLayout(layout)
        self.addItem(secondTab, "Пример формы")
```

Демидовский университет

Действия с QWidget

- show()/hide() — показать/скрыть виджет, аналогично setVisible()
- Перегружаемые функции:
  - enterEvent(event) — курсор вошел на виджет
  - leaveEvent(event) — курсор ушел с виджета
  - mouseMoveEvent(event) — курсор мыши движется по виджету
  - mousePressEvent(event), mouseReleaseEvent(event), mouseDoubleClickEvent(event) — события мыши
  - keyPressEvent(event), keyReleaseEvent(event) — события клавиатуры

Демидовский университет

QMouseEvent — событие мыши

- globalPos(), globalX(), globalY() – позиция курсора относительно окна
- position() – позиция курсора относительно виджета, возвращает QPointF
  - position().x(), position().y() – конкретные координаты

https://doc.qt.io/qtforpython/PySide6/QtGui/QMouseEvent.html

Демидовский университет

Пример QMouseEvent

```
class LogLabel(QLabel):
    eventHappened = Signal(str)
    def __init__(self):
        super().__init__()
        self.setText("Пишу свою историю")
    def enterEvent(self, event):
        self.eventHappened.emit("Ко мне пришел курсор!")
    def leaveEvent(self, event):
        self.eventHappened.emit("Курсор ушёл :(")
    def mousePressEvent(self, event):
        if event.button() == Qt.LeftButton:
            self.eventHappened.emit("На меня нажали левой кнопкой")
        else:
            self.eventHappened.emit("На меня нажали правой кнопкой")
```

Демидовский университет

Drag and Drop — перетаскивание

- QDrag — начало перетаскивания, создаётся в mousePressEvent()
  - Если нужно разделить событие нажатия и событие перетаскивания, то следует переопределить ещё и mouseMoveEvent()
- Функции виджета:
  - setAcceptDrops(True) — разрешить финал перетаскивания
  - dragEnterEvent() — вход курсора с перетаскиванием, здесь можно проверить тип перетаскиваемого объекта
  - dropEvent(event) — отпускание объекта

https://doc.qt.io/qtforpython-6/overviews/qtgui-dnd.html

Демидовский университет

QDrag — перетаскивание

- setMimeData(data) — данные для перетаскивания, например, текст
- setPixmap() — картинка для представления данных во время перетаскивания
- source() — источник, виджет, откуда идет перемещение
- target() — цель, куда в итоге переместились
- supportedActions(), defaultAction() — возможные действия/вариант по умолчанию
  - Qt.CopyAction — скопировать данные в цель
  - Qt.MoveAction — переместить данные в цель
  - Qt.LinkAction — создать ссылку от источника к цели

https://doc.qt.io/qtforpython/PySide6/QtGui/QDrag.html

Пример перетаскивания

```
class DragButton(QPushButton):
    def mouseMoveEvent(self, e):
        if e.buttons() == Qt.LeftButton:
            drag = QDrag(self)
            drag.setMimeData(QMimeData())
            drag.exec(Qt.MoveAction)
class Window(QWidget):
    def __init__(self):
        super().__init__()
        self.setAcceptDrops(True)
        self.layout = QHBoxLayout()
        for l in ['A', 'B', 'C', 'D']:
            btn = DragButton(l)
            self.layout.addWidget(btn)
        self.setLayout(self.layout)
    def dragEnterEvent(self, e):
        e.accept()
    def dropEvent(self, e):
        pos = e.position()
        widget = e.source()
        for n in range(self.layout.count()):
            w = self.layout.itemAt(n).widget()
            if pos.x() < w.x() + w.size().width() // 2:
                self.layout.insertWidget(n-1, widget)
                break
        e.accept()
```

Демидовский университет